

面向 Hybrid TEE-Rollup 仲裁活性的可恢复挑战协议 Recoverable Challenge Protocol for Arbitration Liveness in Hybrid TEE-Rollups

作者：杨帆

摘要：

Hybrid TEE-Rollup 将可信执行环境、链上紧凑提交、数据可用性证明与乐观式挑战结合起来，为元宇宙状态同步、去中心化社交更新和链上 AI 推理等高频 DApp 提供了兼顾低成本与可验证性的设计空间。TEE 可以降低正常执行路径中的验证开销，但不能保证异常路径中的争议一定完成仲裁；欺诈检测能够暴露不一致，却不等价于给出可结算、可罚没或可恢复的最终状态。

本文围绕 Hybrid TEE-Rollup 的异常路径提出可恢复挑战协议（RCP）。协议将挑战会话建模为可恢复状态机，在 OPEN、RESPONDED、NARROWING、TIMED_OUT、RECOVERED、READY_FOR_REPLAY、REPLAYED、RESOLVED 与 SLASHED 等状态之间显式推进。其核心思想是：超时恢复只恢复会话活性，不改写挑战事实；最终仲裁仍由数据可用性绑定、证明哈希、载荷哈希、执行轨迹与重放证据共同决定。

在理论上，本文给出三类不变量：超时恢复不变量要求恢复前后挑战事实集合保持不变；证据一致性不变量要求紧凑提交、DA 记录、证明报告与重放证据指向同一交易语义；数据可用性绑定不变量要求链上 DA 根、载荷身份和字段证明共同约束恢复后的争议上下文。基于这些不变量，本文证明恢复安全性、恢复活性和证据上下文一致性。

实验部分基于研究原型评估协议行为，而非声称生产级系统性能。在建模的超时场景中，无恢复机制时挑战成功率为 0%，引入恢复后挑战成功率为 100%；执行轨迹步数为 4、8、16、32、64、128 时，平均二分定位轮次分别为 2、3、4、5、6、7，与 $\log_2(\text{trace steps})$ 一致；紧凑提交在不同载荷下约为 406 字节；在配置化成本模型中，payload=8192、batch=1000 的 modular DA 配置相对 full-onchain 呈现约 96.33% 的趋势性降本。上述结果仅说明原型机制的可行性与趋势，不代表真实 TEE 安全、生产级 DA、安全主网性能或真实主网费用。

关键词：Hybrid TEE-Rollup；可恢复挑战协议；仲裁活性；挑战事实集合；证据一致性；数据可用性绑定

Abstract:

Hybrid TEE-Rollups combine off-chain trusted execution, compact on-chain commitments, data availability evidence, and optimistic dispute resolution. This design is attractive for high-frequency decentralized applications, but TEE attestation and fraud detection alone do not ensure that an exceptional execution path reaches arbitration. Detecting an inconsistent response is only the beginning of a dispute; the protocol must still preserve evidence, resume progress after timeout, and eventually resolve or slash according to explicit rules.

This paper studies the arbitration-liveness problem in Hybrid TEE-Rollups and proposes a RCP state machine. A timed-out challenge can be recovered by an eligible watcher without changing the underlying dispute facts. The recovered session remains bound to the same data availability root, payload hash, proof hash, execution trace commitment, and replay evidence. Recovery is therefore a liveness operation rather than a correctness oracle or an automatic slashing rule. We formalize the challenge state set, the transition function, the challenge-fact set, and the recovery function, and derive three protocol properties: recovery safety, recovery liveness, and evidence-context consistency. Prototype experiments show that recovery turns modeled timeout cases from detectable-but-stalled disputes into completed arbitration sessions, while preserving logarithmic bisection behavior. Cost results are reported as trends under a configurable model; local EVM gasUsed and Sepolia deployment records are used only as implementation evidence, not as claims of production security or mainnet performance.

Keywords: Hybrid TEE-Rollup; Recoverable Challenge Protocol; Arbitration Liveness; Challenge Facts; Evidence Consistency; Data Availability Binding

1 引言

高频 DApp 对区块链扩容系统提出了双重要求：一方面，系统需要承载大量连续交互，并尽可能降低单次请求的链上成本；另一方面，一旦链下执行出现争议，系统必须保留公开可验证和可追责的异常路径。完全链上执行能够提供直接验证，但成本随请求规模快速放大；完全链下执行能够降低成本，却会削弱用户和外部观察者对执行结果的验证能力。Hybrid TEE-Rollup 试图在两者之间建立折中：使用 TEE 加速正常路径，使用链上提交、DA 证明和挑战机制维持异常路径的可验证性。

然而，Hybrid TEE-Rollup 的异常路径不能只依赖欺诈检测。TEE 证明报告能够绑定输入、输出、随机数和执行环境身份，但无法保证 DA 载荷始终可访问，无法保证证明材料与链上承诺一致，也无法保证被挑战方在争议过程中持续响应。欺诈检测回答的是“是否观察到异常”，而仲裁还需要回答“异常发生后交易如何进入可解释的最终状态”。因此，检测到异常并不等于完成仲裁。

若挑战会话在二分定位、重放或结算之前超时，系统会进入一种不稳定状态：异常已经被观察到，但仲裁尚未完成。直接罚没看似提高活性，却可能在证据尚未闭合时惩罚被挑战方，破坏公平性；重新发起挑战会丢失或弱化原会话中的证据连续性，并扩大攻击者通过反复拖延制造状态碎片的空间；自动终止挑战则将活性失败转嫁给挑战者，使错误提交可能逃逸仲裁。

本文主张将 RCP 作为 Hybrid TEE-Rollup 异常路径中的局部协议机制。恢复操作不裁决正确性，也不直接触发罚没；它只在挑战事实不变的前提下恢复会话推进能力，使争议继续走向二分定位、重放和结算。换言之，恢复机制把“检测到异常但仲裁停滞”的状态转化为“证据上下文不变且可继续推进”的状态。

本文贡献如下。第一，提出可恢复挑战状态机，将挑战从系统实现流程提升为具有明确安全语义和活性目标的协议机制。第二，形式化挑战事实集合、恢复函数与恢复不变性，说明恢复操作为何不会改写仲裁依据。第三，围绕数据可用性根、载荷哈希、证明哈希和重放证据建立证据一致性与 DA 绑定规则。第四，基于研究原型评估超时恢复、二分定位、紧凑提交、故障语义、本地 EVM gasUsed 与 Sepolia 可部署性，并明确结果边界。

2 背景与相关工作

Optimistic Rollup 通过乐观执行和欺诈证明降低正常路径成本，其安全性依赖挑战窗口内至少一个诚实参与者发现并推动争议。传统 fraud proof 关注错误状态转移能否被证明，强调提交、挑战和结算的安全边界。该路线为本文提供了异常路径的基本框架，但多数讨论主要围绕错误检测和状态正确性展开，对挑战会话超时后的仲裁恢复问题着墨较少。

Arbitrum 的交互式欺诈证明展示了二分定位在长执行轨迹争议中的有效性：双方逐步缩小争议区间，最终将错误定位到单步指令或小型执行片段。Optimism fault proof 体系则强调可替换的故障证明虚拟机、争议游戏和模块化证明路径。这些工作说明交互式争议定位能够降低链上重放负担，但它们的核心问题仍是如何证明错误，而不是在挑战已经打开但会话超时后如何恢复仲裁活性。

opML、Cartesi 等工作将乐观式验证扩展到机器学习推理、通用计算或应用特定执行环境。opML 通常采用二分定位和单步仲裁，使大规模计算结果在争议时具备可验证路径；Cartesi 通过链下 Linux 环境和链上争议机制支持复杂应用。这些工作与本文共享“正常路径链下执行、异常路径链上仲裁”的结构，但本文关注的是仲裁流程本身的可恢复性，而不是完整 FPVM、真实 ML 证明或通用链下计算平台。

数据可用性研究为挑战恢复提供证据基础。LazyLedger、Celestia、EigenDA 以及 EIP-4844 等方向强调将数据发布、可用性采样或 blob 数据成本从执行验证中解耦。对于 Hybrid TEE-Rollup，DA 不仅服务于扩容，还决定恢复后的挑战能否重新加载同一争议载荷和证明上下文。若恢复后无法证明其仍指向原始 DA 根和载荷哈希，恢复机制就可能把活性操作变成证据替换入口。

TEE-assisted blockchain execution 研究利用可信执行环境降低链下执行的验证成本。TEEROLLUP 等工作表明异构 TEE 可以改善 Rollup 验证效率；Optimistic TEE-Rollups 等工作探索 TEE 与乐观验证的结合。这些研究强调快速路径效率和 TEE 证明绑定，但 TEE 不能替代异常路径仲裁。本文在这些工作的基础上进一步指出：已有工作多关注错误检测、执行验证或数据可用性发布，较少将“挑战超时后的仲裁恢复”作为独立协议问题处理。

表 1 相关工作与本文差异

方向	已有关注	与本文关系	仍缺口
Optimistic Rollup fraud proof	错误状态转移检测与挑战窗口	提供异常路径基本框架	较少讨论挑战已打开后的超时恢复
Arbitrum interactive proof	二分定位与单步仲裁	提供对数级争议定位思想	恢复语义通常不是独立机制
Optimism fault proof	争议游戏与 FPVM	说明故障证明可模块化	本文不实现完整 FPVM，而研究恢复活性
opML / Cartesi	链下复杂计算的乐观验证	共享链下执行、链上争议结构	重点不在 TEE-Rollup 超时仲裁恢复
EigenDA / Celestia / LazyLedger / EIP-4844	数据发布与可用性成本	为恢复后的证据重载提供基础	不直接规定挑战恢复状态机
TEE-assisted execution	TEE 证明与正常路径加速	为 Hybrid TEE-Rollup 提供执行组件	TEE 不能保证异常仲裁完成

2.1 Rollup Fraud Proof

Rollup fraud proof 研究主要围绕错误状态转移的可发现性和可验证性展开。Arbitrum Nitro 将交互式争议游戏与 Nitro 执行栈结合，通过链下执行和链上断言降低常规路径成本；OP Stack 则将 fault proof、争议游戏和模块化执行栈作为可组合基础设施。zkSync 与 StarkNet 代表的 validity rollup 路线通过有效性证明减少欺诈挑战需求，但其核心同样是状态正确性证明。上述系统为链下执行的正确性和验证性提供了重要基础，但并不直接回答 challenge session 在超时后如何恢复仲裁活性。

2.2 Interactive Verification

交互式验证通过多轮缩小争议范围，将长执行轨迹上的错误定位到单步或小范围执行片段。Arbitrum、Optimism fault proof、opML 和 Cartesi Whitepaper 均体现了这一思想：通过 bisection 或应用特定 VM 降低链上重放成本。本文继承交互式验证的定位结构，但将研究重点从“如何定位错误”推进

到“定位过程停滞后如何在不改变 挑战事实集合 的前提下恢复仲裁”。

2.3 Data Availability

Data availability 方向关注完整数据能否被验证者获得。Celestia、EigenDA、Espresso DA、LazyLedger 与 EIP-4844 从不同角度讨论数据发布、采样、排序和 blob 成本结构；Sovereign Rollup 进一步将执行环境、结算和 DA 之间的边界重新划分。本文并不替代这些 DA 系统，而是把 DA Binding 作为 RCP 的前提：恢复后的会话必须仍由同一 da_merkle_root、payload_hash 与字段证明约束。

2.4 TEE-assisted Execution

TEE-assisted execution 研究使用 SGX、TDX 或其他可信执行环境降低正常路径验证成本。TEEROLLUP 和 Optimistic TEE-Rollups 表明 TEE 可与 Rollup 或 optimistic verification 结合，为高频计算提供快速路径。然而，TEE attestation 只能提供执行摘要绑定，不能保证争议过程完成、DA 数据可访问或被挑战方持续响应。本文因此将 TEE 定位为正常路径优化组件，而将 仲裁活性 交由 RCP 状态机处理。

2.5 仲裁活性

已有工作多强调 correctness 与 verifiability，即错误是否能够被证明、状态是否能够被验证、数据是否能够被获得。本文关注的仲裁活性是互补性质：当错误已经可见或挑战已经打开时，协议是否能够在超时、中断或证据重载之后继续推进到 RESOLVED 或 SLASHED。RCP 的贡献在于把这一问题显式形式化为局部状态机机制，而不是把超时简单解释为挑战失败、自动罚没或重新发起。

表 2 现有方案与 RCP 的能力对比

Interactive Proof	TEE Support	Recovery	Fact Preservation
✓	✗	✗	✗
✓	✗	✗	✗
✓	✗	✗	✗
✓	✗	✗	✗
✓	✓	✓	✓

现有工作主要关注错误检测、交互式争议定位或执行验证。而本文提出的 RCP 将恢复机制作为

独立协议组件进行形式化建模，并通过 **Fact Preservation** 保证恢复不会改变仲裁依据，从而突出其在仲裁活性恢复和挑战事实保持方面的协议创新。

本文真正解决的问题并非单纯 **Recovery**，而是 **Arbitration Liveness**。**Recovery** 是实现仲裁活性的协议手段，挑战恢复能力服务于仲裁活性目标，而不是一个孤立的工程补丁。

3 问题定义与威胁模型

本文将 Hybrid TEE-Rollup 中的一笔可挑战交易表示为 $tx = (\text{commit}, \text{da_entry}, \text{attest}, \text{trace}, \text{status})$ 。其中 **commit** 为链上紧凑提交，**da_entry** 保存链下载荷与字段级证明，**attest** 绑定输入、输出和执行环境，**trace** 表示争议定位与重放所需执行轨迹，**status** 表示交易或挑战会话状态。

协议目标是仲裁活性（**Arbitration Liveness**）：一个在争议窗口内打开的合法挑战，不应因为被挑战方失联、会话超时或中间状态停滞而永久无法结算。在证据可恢复且至少一个诚实看护者可用的条件下，挑战应继续推进至 **RESOLVED** 或 **SLASHED**，而不是停留在 **OPEN**、**RESPONDED**、**NARROWING**、**TIMED_OUT**、**RECOVERED**、**READY_FOR_REPLAY** 或 **REPLAYED** 等中间状态。

3.1 协议假设

- 链上合约提供有序状态转移，并记录挑战会话的关键状态、期限和证据摘要。
- 至少一个诚实挑战者或看护者能够观察链上提交，并在超时后提交恢复交易。
- 原型中的 **DA** 层提供可验证注册表、载荷哈希、Merkle 根和字段证明，但不声称具备生产级 **DA** 经济安全。
- 原型中的 **TEE** 为模拟证明报告，只建模输入输出绑定和环境身份绑定，不声称真实硬件安全。
- 二分定位与重放依赖原型生成的预期轨迹和声称轨迹，不等价于完整链上 **FPVM**。

3.2 对手模型

攻击者可控制提交者、排序者或被挑战方，并尝试提交错误 **response**、篡改 **DA payload**、提供无效 **attestation**、隐藏数据可用性材料、破坏 **Merkle proof**、制造 **trace length mismatch**，或在挑战过程中停

止响应。攻击者不能破坏哈希抗碰撞性，不能伪造建模验证器认可的有效证明，也不能改写已经记录的链上状态。本文重点处理攻击者通过拒绝响应或超时制造仲裁活性失败的情况。

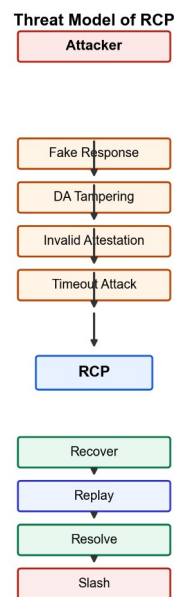


Figure 1 Threat Model of RCP

3.3 RCP 的必要性分析

异常路径不能只回答“错误是否被检测到”。协议还必须规定检测之后如何继续仲裁、何时允许恢复、何时允许重放、何时可以结算或罚没。下表比较几类直观方案。

表 3 仲裁恢复方案对比

方案	仲裁活性	公平性	证据连续性	安全语义清晰度	实现复杂度
无恢复机制	低：超时后会话可能停滞	中：不额外惩罚但错误可能逃逸	较强但无法继续利用	检测与仲裁边界模糊	低
超时直接罚没	高：能迅速终结	低：证据未闭合时可能误罚	中：不需要继续证据链	将活性失败等同作恶，语义过强	中

方案	仲裁活性	公平性	证据连续性	安全语义清晰度	实现复杂度
重新发起挑战	中：依赖新会话成功	中：给双方重新组织机会	低：原会话证据和期限被切断	存在重置争议上下文风险	中
自动终止挑战	低：异常可能回到未仲裁状态	偏向被挑战方	中：证据仍存在但流程结束	把超时解释为挑战失败，语义偏置	低
本文可恢复挑战	高：超时后可继续推进	高：恢复不直接裁决或罚没	高：保持 Facts(cs) 不变	清晰：恢复是活性操作	中高

3.4 形式化定义

挑战状态集合定义为 $S = \{\text{OPEN}, \text{RESPONDED}, \text{NARROWING}, \text{TIMED_OUT}, \text{RECOVERED}, \text{READY_FOR_REPLAY}, \text{REPLAYED}, \text{RESOLVED}, \text{SLASHED}\}$ 。其中 OPEN、RESPONDED 与 NARROWING 表示争议仍在交互式定位阶段；TIMED_OUT 表示会话期限已过但尚未完成仲裁；RECOVERED 与 READY_FOR_REPLAY 表示恢复后重新具备推进条件；REPLAYED 表示单步重放已产生可结算结果；RESOLVED 与 SLASHED 为终止状态。

协议事件集合 E 包括 open、respond、step、timeout、recover、prepare_replay、replay、resolve 与 slash。状态转移函数记为 $\delta: S \times E \rightarrow S$ 。例如 $\delta(\text{NARROWING}, \text{timeout}) = \text{TIMED_OUT}$ ， $\delta(\text{TIMED_OUT}, \text{recover}) = \text{RECOVERED}$ ， $\delta(\text{RECOVERED}, \text{prepare_replay}) = \text{READY_FOR_REPLAY}$ ， $\delta(\text{READY_FOR_REPLAY}, \text{replay}) = \text{REPLAYED}$ ， $\delta(\text{REPLAYED}, \text{resolve}) = \text{RESOLVED}$ 或 $\delta(\text{REPLAYED}, \text{slash}) = \text{SLASHED}$ 。非法事件不改变状态或触发合约拒绝。

对挑战会话 cs，挑战事实集合 F(cs) 表示最终仲裁所依赖的不可变事实，包括 commit、da_merkle_root、payload_hash、proof_hash、attestation

binding、expected trace commitment、claimed trace commitment、已记录 mismatch、replay evidence hash 与打开挑战时的争议上下文。恢复函数 $\text{Recover}(cs)=cs'$ 只更新恢复者、恢复时间、新的推进期限和会话活性标记，不更新 $F(cs)$ 。因此恢复不变性写作 $F(cs)=F(\text{Recover}(cs))$ 。该等式是本文协议语义的核心：恢复不是证据替换，也不是裁决函数，而是保持挑战事实不变的状态推进。

3.5 符号说明

表 4 Symbol Definitions

Symbol	Description
cs	Challenge Session
S	State Set
E	Event Set
δ	State Transition Function
$F(cs)$	Challenge Facts
$\text{Recover}(cs)$	Recovery Function
da_merkle_root	Data Availability Merkle Root
payload_hash	Payload Hash
proof_hash	Proof Hash

为便于后文描述，本文将挑战会话记为 $cs = (id, s, F, T)$ ，其中 id 表示会话标识， s 表示当前状态， F 表示挑战事实集合， T 表示超时与恢复时间窗参数。

状态机定义为 $M = (S, E, \delta)$ ，其中 S 为状态集合， E 为事件集合， $\delta: S \times E \rightarrow S$ 为状态转移函数。

挑战事实集合定义为 $F(cs) = \{\text{commit}, \text{da_merkle_root}, \text{payload_hash}, \text{proof_hash}, \text{trace_commitment}, \text{replay_evidence_hash}\}$ 。恢复函数记为 $\text{Recover}(cs) = cs'$ ，其核心不变性要求 $F(cs) = F(\text{Recover}(cs))$ 。

4 系统概览

原型采用执行层、数据层和验证层三层结构。执行层生成链下响应和模拟 TEE 证明；数据层保存完整 payload、证明报告、字段级 Merkle proof 和 DA 状态；验证层在链上表达 compact commit、challenge open、respond、step、recover、replay 和 resolve 等状态转移。该架构旨在研究异常路径仲裁机制，而不是构造完整生产级 Rollup。

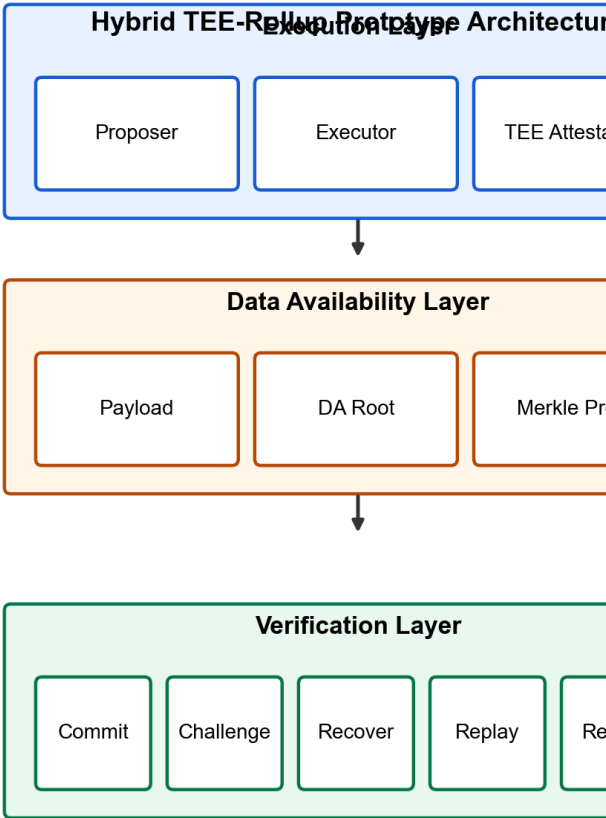


图 2 可恢复挑战驱动的 Hybrid TEE-Rollup 原型架构

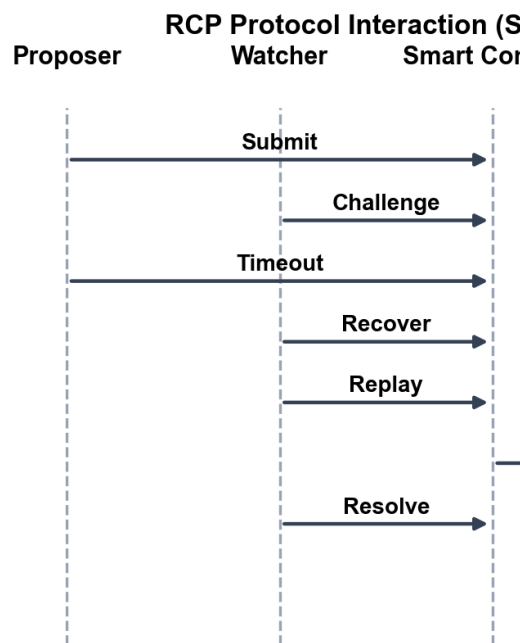


图 3 RCP 协议交互时序

正常路径中，提交者只向链上提交 `state_root`、`output_hash`、`proof_hash`、`da_pointer` 和 `da_merkle_root` 等紧凑承诺。异常路径中，挑战者通过 `da_pointer` 加载完整 `payload` 和字段证明，通过 `proof_hash` 验证证明报告身份，并在必要时通过二分定位和单步重放判断执行差异。若会话超时，看护者调用 `recover`，使会话返回可推进状态，但不得提交新的挑战事实。

5 可恢复挑战协议

可恢复挑战协议的目标，是将 `challenge` 从一次性欺诈检测流程扩展为具备恢复语义的仲裁状态机。其设计原则包括：第一，恢复只处理活性，不处理正确性；第二，恢复后必须沿用同一挑战事实集合；第三，结算仍由证据一致性、DA 绑定和重放结果决定；第四，终止状态不可恢复，避免已结算争议被再次打开。

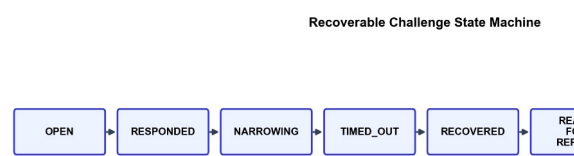


图 4 可恢复挑战协议状态机

5.1 状态转移与恢复路径

表 5 RCP 状态转移

当前状态	事件	下一状态	注
OPEN	respond	RESPONDED	1
RESPONDED	step	NARROWING	2
NARROWING	timeout	TIMED_OUT	3
TIMED_OUT	recover	RECOVERED	4
RECOVERED	prepare_replay	READY_FOR_REPLAY	5
READY_FOR_REPLAY	replay	REPLAYED	6
REPLAYED	resolve	RESOLVED	7
REPLAYED	slash	SLASHED	8

表注：状态转移语义：1. OPEN -> respond -> RESPONDED: 被挑战方提交回应; 2. RESPONDED -> step -> NARROWING: 进入二分定位; 3. NARROWING -> timeout -> TIMED_OUT: 会话超时但未完成仲裁; 4. TIMED_OUT -> recover -> RECOVERED: 看护者恢复会话活性; 5. RECOVERED -> prepare_replay -> READY_FOR_REPLAY: 争议区间已可重放; 6. READY_FOR_REPLAY -> replay -> REPLAYED: 执行单步重放并记录结果; 7. REPLAYED -> resolve -> RESOLVED: 挑战不成立或完成普通结算; 8. REPLAYED -> slash -> SLASHED: 重放失败且满足罚没条件;

恢复的合法条件包括：会话尚未进入 RESOLVED 或 SLASHED；当前时间超过 timeout_at；原挑战在争议窗口内打开；commit、DA 记录和已记录证据仍可定位；恢复调用只携带恢复元数据，而不携带新的争议事实。若任一条件不满足，恢复请求应被拒绝。

5.2 二分定位与重放

当争议涉及执行轨迹时，协议通过二分定位缩小 expected trace 与 claimed trace 的差异区间。对于长度为 n 的轨迹，理想轮次接近 $\log_2(n)$ 。恢复机制不重置二分区间，也不选择新的目标步；它只允许已停滞的会话继续按照原有区间推进。区间缩小到单步后，协议进入 READY_FOR_REPLAY，并通过 replay 比较预期状态哈希与声称状态哈希。

5.3 恢复语义

Recover(cs)=cs' 的作用是刷新会话活性状态，并记录恢复者身份、恢复时间和新的推进期限。它不修改 commit、DA 根、载荷哈希、证明哈希、轨迹承诺、mismatch 标记或 replay evidence hash。恢复后的

会话如果继续重放并发现不一致，罚没依据来自重放和证据一致性检查，而不是来自 recover 本身。

Algorithm 1 Recover Challenge Procedure

```

Input: challenge session cs
Output: recovered session cs'
1: require cs.status not in {RESOLVED, SLASHED}
2: require now > cs.timeout_at
3: require EvidenceAvailable(cs.commit,
  cs.da_entry, cs.evidence)
4: facts_before <- Facts(cs)
5: cs'.status <- RECOVERED
6: cs'.recovery.watcher <- msg.sender
7: cs'.recovery.recovered_at <- now
8: cs'.timeout_at <- now + recovery_window
9: require Facts(cs') = facts_before
10: return cs'

```

Algorithm 2 Interactive Challenge Resolution

```

Input: challenge session cs, challenger evidence ev
Output: terminal state RESOLVED or SLASHED
1: cs.status <- OPEN
2: cs <- Respond(cs)
3: while DisputeInterval(cs) > 1 do
4:   cs <- StepBisection(cs)
5:   if Timeout(cs) then cs <- Recover(cs)
6: end while
7: cs.status <- READY_FOR_REPLAY
8: replay_result <- ReplaySingleStep(cs.target_step, ev)
9: cs.status <- REPLAYED
10: if replay_result is inconsistent then
11:   cs.status <- SLASHED
12: else
13:   cs.status <- RESOLVED
14: end if
15: return cs.status

```

Algorithm 3 Evidence Consistency Verification

```

Input: commit c, DA entry da, attestation att,
replay evidence re
Output: accept or reject
1: require c.da_merkle_root = da.merkle_root
2: require Hash(da.payload) = c.payload_hash
3: require VerifyMerkle(da.field_proofs,
c.da_merkle_root)
4: require Hash(att) = c.proof_hash
5: require att.input_hash and att.output_hash match
da.payload fields
6: require Hash(re) = c.replay_evidence_hash
7: require re.target_step =
ChallengeTargetStep(c.challenge_id)
8: return accept

```

5.4 协议性质证明

引理 1 (Fact Preservation)。对任意满足恢复前置条件的挑战会话 cs ，均有 $F(cs)=F(Recover(cs))$ 。

证明。Recover(cs) 仅更新 status、recovery.watcher、recovered_at 与 timeout_at 等会话活性元数据。挑战事实集合中的 commit、da_merkle_root、payload_hash、proof_hash、轨迹承诺、目标步、mismatch 标记与 replay evidence hash 均不在恢复写集中。因此恢复前后挑战事实集合相等。

引理 2 (Replay Target Preservation)。恢复不会改变二分定位得到的目标步。

证明。目标步由 NARROWING 阶段的争议区间收缩过程决定。Recover(cs) 不修改区间端点、二分轮次、expected trace commitment 或 claimed trace commitment，因此由这些值派生的 replay target 在恢复前后保持不变。

引理 3 (Evidence Binding Preservation)。恢复不会改变 payload_hash、proof_hash、da_merkle_root 与 replay evidence hash，因此证据上下文保持一致。

证明。上述四类哈希均属于挑战事实集合，并由链上提交或已记录证据锚定。恢复操作没有写入这些字段的权限；任何恢复后替换载荷、证明或重放证据的尝试都会破坏哈希或 Merkle 验证关系。故证据绑定在恢复前后保持不变。

基于上述三个引理，以下定理分别刻画 RCP 的安全性、活性与证据上下文一致性。引理 1 保证恢复不改写仲裁事实，引理 2 保证恢复不改变重放目标，引理 3 保证恢复后的证据仍绑定到同一争议上下文。

定理 1 (恢复安全性)。在 $F(cs)=F(Recover(cs))$ 且证据一致性检查有效的前提下，恢复操作不会改变挑战的最终仲裁结果。

证明。最终仲裁结果由挑战事实集合中的 commit、DA 根、payload_hash、proof_hash、轨迹承诺、mismatch 与 replay evidence 决定。恢复操作只更新会话活性元数据，不修改上述事实。由于证据一致性检查有效，恢复后的重放与结算仍读取同一组仲裁输入。因此，若恢复前某一事实集合将导向 RESOLVED 或 SLASHED，恢复后使用相同事实集合得到的裁决条件不变。故恢复不改变仲裁结果。

定理 2 (恢复活性)。在至少一个诚实看护者可且必要证据可恢复的条件下，超时挑战不会永久停留在 TIMED_OUT 或其他中间状态，而可以继续推进至 RESOLVED 或 SLASHED。

证明。挑战进入 TIMED_OUT 后，诚实看护者能够观察链上会话并提交 recover。若证据可恢复，恢复合法性检查通过，状态转移至 RECOVERED，随后根据原二分区间进入 READY_FOR_REPLAY、REPLAYED，并由 resolve 或 slash 进入终止状态。由于终止状态不可恢复，推进过程不会循环回超时状态。故在调度最终执行恢复交易且证据可用的条件下，会话不会永久停留在中间状态。

定理 3 (证据上下文一致性)。当数据可用性根、载荷哈希、证明哈希和重放证据被绑定后，恢复后的挑战仍然指向同一争议上下文。

证明。DA 根约束字段级 Merkle proof，payload_hash 约束完整载荷身份，proof_hash 约束证明报告身份，replay evidence hash 约束单步重放材料。恢复函数不修改这些绑定项。若攻击者试图在恢复后替换载荷、证明或重放材料，则至少会破坏上述哈希或 Merkle 关系之一，从而被证据一致性检查拒绝。因此，恢复后的挑战仍指向原始争议上下文。

5.5 Complexity Analysis

$T_{\text{recover}}(n) = O(1)$; $T_{\text{bisection}}(n) = O(\log n)$;
 $T_{\text{replay}}(n) = O(1)$; $S_{\text{onchain}} = O(1)$; $S_{\text{offchain}} = O(n)$ 。

其中，恢复操作只检查已记录的会话状态、超时条件和挑战事实摘要，因而 $T_{\text{recover}}(n)$ 与轨迹长度 n 无关。二分定位每轮将争议区间减半，因而轮次为 $O(\log n)$ 。单步 replay 只验证目标步的状态哈希和证据一致性，因此 $T_{\text{replay}}(n)=O(1)$ 。链上

只保存紧凑提交、DA 根和会话状态，即 $S_{onchain}=O(1)$ ；完整 payload 与 trace 保存在链下，即 $S_{offchain}=O(n)$ 。

恢复操作的复杂度为 $O(1)$ 。`Recover(cs)` 只检查会话终止状态、超时条件和证据入口是否存在，并更新固定数量的状态元数据；其成本不随 trace length 或 payload size 增长。

二分定位复杂度为 $O(\log n)$ ，其中 n 为 trace length。每一轮 step 将争议区间缩小约一半，因此定位到单步目标所需轮次数量与执行轨迹长度的对数成正比。

单步重放复杂度为 $O(1)$ 。在 `READY_FOR_REPLAY` 状态下，协议只对二分定位得到的 target step 执行一次局部重放或哈希比较；该操作不扫描完整轨迹。

存储复杂度分为链上与链下两部分。链上存储为 $O(1)$ ，因为合约仅保存 compact commit、状态元数据、DA root、证明哈希和重放证据哈希等固定大小锚点。链下存储为 $O(n)$ ，因为完整执行轨迹、payload 与证明材料随 trace length 或业务载荷规模增长。该分离是 evidence-carrying compact commit 能够兼顾链上低成本与恢复后证据重载的原因。

6 证据携带型提交与数据可用性绑定

证据携带型紧凑提交（evidence-carrying compact commit）不是单纯的降本摘要，而是异常路径的证据入口。链上提交记录 state_root、output_hash、proof_hash、da_pointer 与 da_merkle_root；DA entry 保存完整 prompt、response、attestation、payload_hash、leaf hashes、Merkle root 与字段级证明。

数据可用性绑定要求 `commit.da_merkle_root` 等于 DA entry 中的 Merkle root，且被挑战字段能够在该根下通过字段证明验证。证明报告中的输入输出哈希必须与 payload 字段一致，证明报告自身哈希必须等于 proof_hash，重放证据必须与挑战会话中记录的轨迹承诺一致。只有这些关系同时成立，恢复后的会话才被视为仍连接到同一证据上下文。

该设计将链上成本和证据可恢复性分离：链上保留固定大小的仲裁锚点，完整载荷保存在 DA 层；当挑战发生或恢复后需要继续仲裁时，验证者通过 DA 根和字段证明重新加载同一争议材料。

7 实验评估

本节将实验解释为协议机制评估，而不是生产系统性能基准。所有实验均基于研究原型：TEE 为模拟环境，DA 为可验证注册表，成本为配置化模型，本地 EVM gasUsed 只表示 Hardhat 本地链上的操作开销，Sepolia 部署只说明合约可部署性。

7.1 实验设置

实验包括 3200 条成本实验记录、600 条挑战实验记录和 800 条失败场景实验记录。成本实验覆盖 payload size 128、512、2048、8192，prompt length 64、256，batch size 1、10、100、1000，以及 full_onchain_calldata、compact_external_da、compact_eip4844_like、compact_modular_da_sampling 四类 DA profile。挑战实验覆盖 trace steps 4、8、16、32、64、128，challenge timeout=2，max bisection rounds=16。失败场景包括 normal、attestation_invalid、da_unavailable、da_proof_invalid、response_tampered、trace_length_mismatch 与 challenge_timeout_no_recover 和 challenge_timeout_recover。

表 6 Baseline 对比

Baseline	处理方式	预期问题	本文比较目的
No Recovery	检测超时但不恢复	仲裁停滞	衡量恢复对活性的贡献
Restart Challenge	重新打开挑战	证据连续性弱化	说明重启不是等价恢复
Auto Slash	超时直接罚没	公平性风险	说明罚没不能替代仲裁
Ours: Recoverable Challenge	保持事实不变并恢复推进	实现复杂度较高	验证局部恢复机制的活性与语义边界

7.1.1 Baseline Discussion

No Recovery 基线将 timeout 视为可观察异常，但不提供后续推进路径。因此它能够保留一定证据连续性，却无法保证仲裁活性；一旦对手通过停止响应制造中间状态，协议只能记录失败而无法完成仲裁。

Restart Challenge 基线通过重新打开会话恢复表面活性，但其问题在于新会话与旧会话之间需要额外证

明同一 挑战事实集合。若缺乏严格绑定，重启会弱化证据连续性，并允许攻击者通过反复拖延制造多个局部争议上下文。

Auto Slash 基线具有最高的终止速度，但把 timeout 直接解释为可罚没行为会扩大惩罚语义。对于网络延迟、看护者调度失败或证据尚未闭合的情况，自动罚没可能破坏公平性，并把 liveness failure 错误转化为 safety decision。

RCP 在三类基线之间取得更细粒度的协议边界：它像 Restart Challenge 一样恢复推进能力，但不切断原会话事实；它像 Auto Slash 一样避免永久停滞，但不把恢复本身作为惩罚依据；它比 No Recovery 多提供 O(1) 的状态恢复路径，使超时会话能够继续进入 replay 与 settlement。

7.2 RQ1: 恢复机制是否保持仲裁活性

本 RQ 使用 Modeled Timeout Scenario 说明协议在超时建模场景下的状态推进行为，不表示开放网络中的统计结果。

Research Question：在建模的挑战超时场景中，RCP 是否能将“已检测但停滞”的会话推进到完成仲裁。

Hypothesis：若恢复保持挑战事实不变并恢复会话推进期限，则超时挑战可以继续进入 replay 与 resolve/slash。

Metrics：detection_rate、challenge_success_rate、slashed_rate 与 recovery_success_rate。

表 7 RQ1 Modeled Timeout Scenario 中恢复机制对仲裁活性的影响

场景	检测率	挑战成功率	罚没率	注
Timeout-NoRecover	100.00 %	0.00%	0.00%	1
Timeout-Recover	100.00 %	100.00 %	100.00 %	2

表注：RQ1 场景解释：1. challenge_timeout_no_recover -> 100.00% -> 0.00% -> 0.00%: 异常可被观察，但仲裁未完成；2. challenge_timeout_recover -> 100.00% -> 100.00% -> 100.00%: 建模场景中恢复后完成重放与罚没；

Results：无恢复机制时，超时可被检测但挑战成功率为 0%；引入恢复后，在该建模超时场景中挑战成功率与罚没率为 100%。

Analysis：结果说明 recover 的价值是仲裁活性，而不是检测能力。检测率在两个场景中均为 100%，差异来自恢复是否允许会话继续进入重放和结算。

Limitation：100% 挑战成功率只在当前建模超时场景、证据可恢复和原型故障注入条件下成立，不能外推为真实网络中的通用成功率。

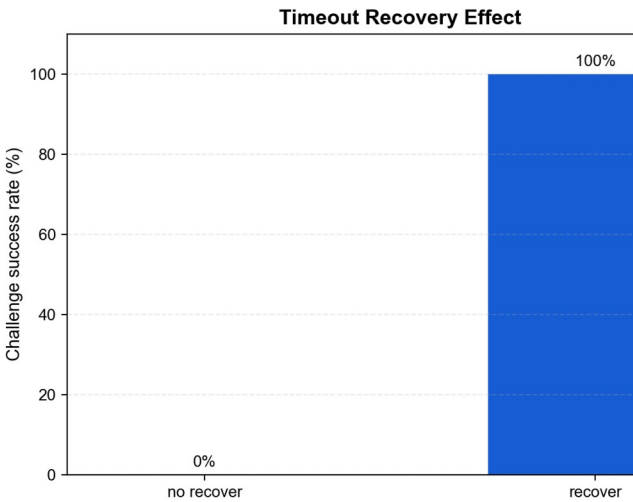


图 5 恢复对超时场景挑战完成率的影响

7.3 RQ2: 二分定位是否保持对数级争议定位

Research Question：恢复机制是否破坏交互式挑战的二分定位结构。

Hypothesis：如果恢复不重置争议区间，则二分定位轮次仍应接近 log2(trace steps)。

Metrics：avg_bisection_rounds 与理论 log2 轮次。

表 8 RQ2 二分定位轮次

Trace steps	平均二分轮次	理论 log2 轮次	挑战成功率
4	2.00	2.00	100.00%
8	3.00	3.00	100.00%
16	4.00	4.00	100.00%
32	5.00	5.00	100.00%
64	6.00	6.00	100.00%

Trace steps	平均二分轮次	理论 log2 轮次	挑战成功率
128	7.00	7.00	100.00%

Results: 执行轨迹步数为 4、8、16、32、64、128 时, 平均二分轮次分别为 2、3、4、5、6、7, 与理论 log2 轮次一致。

Analysis: 恢复操作没有引入线性扫描, 也没有重置已缩小的争议区间, 因此交互式定位的渐近结构得到保留。

Limitation: 当前重放证据由原型逻辑提供, 不能证明完整 FPVM 或真实链上执行验证的复杂度。

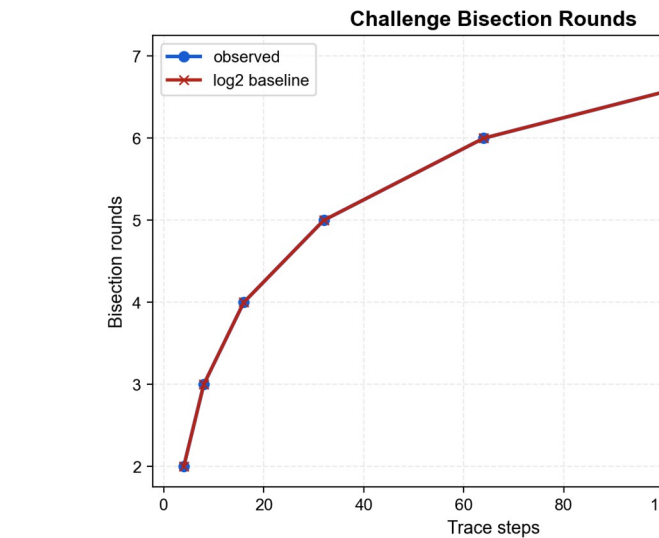


图 6 二分挑战轮次与 trace steps 的关系图 6 二分挑战轮次与 trace steps 的关系

7.4 RQ3: 紧凑提交是否提供稳定证据锚点

Research Question : evidence-carrying compact commit 是否能在载荷规模增长时保持近似固定的链上提交规模。

Hypothesis: 若链上只提交固定摘要和 DA 指针, 则 compact bytes 应基本不随完整 payload 增长。

Metrics : full_bytes 、 compact_bytes 、 byte_reduction_ratio。

表 9 RQ3 紧凑提交字节规模

Payload	Full bytes (示例)	Compact bytes	字节降幅
128	1682.84 / 1876.84	406.00	75.33% / 77.98%
512	3986.84 / 4180.84	406.00	89.39% / 89.92%
2048	13204.84 / 13398.84	406.00	96.76% / 96.81%
8192	50068.84 / 50262.84	406.00	99.14%

Results: compact commit 在不同载荷规模下约为 406 字节, 而 full payload bytes 随 payload size 增长显著上升。

Analysis: 结果说明紧凑提交可作为稳定证据锚点, 将链上提交规模从完整业务载荷中解耦。该结论服务于恢复机制: 恢复后会话可以通过固定锚点重新定位 DA 证据。

Limitation: 该结果基于 canonical JSON 与原型数据结构, 不等同于真实 ABI 编码、calldata、storage layout 或真实 TEE 证明大小。

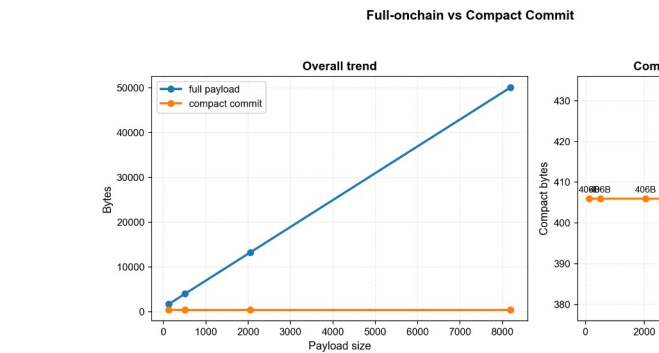


图 7 full payload 与 compact commit 的字节规模对比图 7 full payload 与 compact commit 的字节规模对比

7.5 RQ4: DA 绑定带来怎样的成本趋势

Research Question : 在配置化成本模型中, compact commit + DA 是否呈现相对 full-onchain 的摊销成本下降趋势。

Hypothesis: 随着 batch size 增大, 固定提交开销被摊薄; 随着 payload size 增大, full-onchain 的相对成本上升, compact + DA 的趋势优势更明显。

Metrics: amortized_gas 与 reduction_ratio。

表 10 RQ4 DA 成本趋势

Paylo ad	Prom pt	Full-onchain 单笔 gas	Modular DA 单笔 gas	注
8192	64	801101.44	29026.98	1
		(batch=1) /	(batch=1) /	
		801.10	29.03	
		(batch=1000	(batch=1000	
		)	)	
8192	256	804205.44	29114.28	2
		(batch=1) /	(batch=1) /	
		804.21	29.11	
		(batch=1000	(batch=1000	
		)	)	

表注: DA 成本趋势说明: 1. 8192 -> 64 -> 801101.44 (batch=1) / 801.10 (batch=1000) -> 29026.98 (batch=1) / 29.03 (batch=1000): 96.33%; 2. 8192 -> 256 -> 804205.44 (batch=1) / 804.21 (batch=1000) -> 29114.28 (batch=1) / 29.11 (batch=1000): 96.33%;

Results : 在 payload=8192 的配置下, compact_modular_da_sampling 相对 full_onchain_calldata 呈现约 96.33% 的趋势性降本。batch size 从 1 增至 1000 时, 各路线的单笔摊销 gas 均下降。

Analysis: 96.33% 反映给定 cost model 与 DA profile 下的结构性趋势, 而非主网费用结论。它说明 DA 绑定和紧凑提交可以在高频场景中共同降低证据携带路径的摊销压力。

Limitation: 模型未包含实时 calldata 定价、blob 市场价格、DA 网络服务费、拥堵和争议率波动; 本地 EVM gasUsed 也不能等同于主网成本。

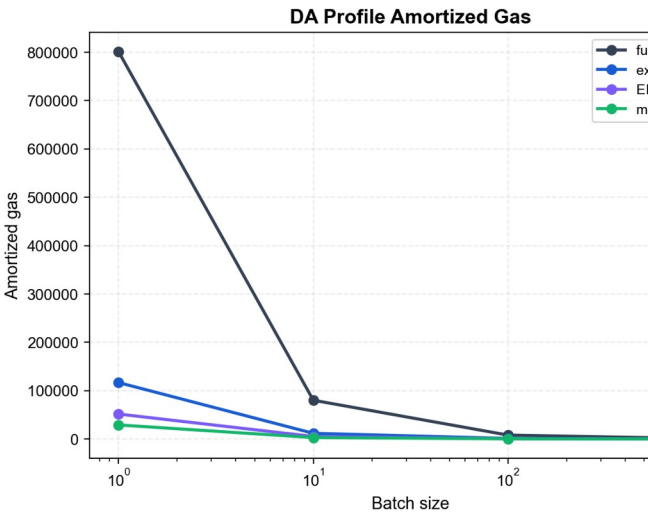


图 8 不同 DA profile 下的摊销 gas 趋势图 8 不同 DA profile 下的摊销 gas 趋势

7.6 RQ5: 故障语义如何影响仲裁完成

Research Question: 不同故障是否都应被解释为同一种罚没结果。

Hypothesis: 协议应区分检测、拒绝、恢复、重放、结算和罚没, 不能把所有可检测异常都写成已完成惩罚。

Metrics: detection_rate、challenge_success_rate、slashed_rate。

表 11 RQ5 故障语义与仲裁结果

类别	场景	检测率	挑战成功/罚没	注
正常路径	normal	0.00%	0.00% / 0.00%	1
可检测但未统一罚没	Att-Invalid, DA-Unavailable, DA-Proof-Invalid	100.00 %	0.00% / 0.00%	2
挑战与罚没闭环	Resp-Tampered, Trace-Mismatch	100.00 %	100.00% / 100.00%	3

类别	场景	检测率	挑战成功/罚没	注
超时	no_recover vs recover	100.00	0.00%	4
活性		%	到	
对照			100.00%	

表注：故障语义说明: 1. 正常路径 -> normal -> 0.00% -> 0.00% / 0.00%: 未产生误报; 2. 可检测但未统一罚没 -> attestation_invalid, da_unavailable, da_proof_invalid -> 100.00% -> 0.00% / 0.00%: 当前原型主要体现检测或拒绝; 3. 挑战与罚没闭环 -> response_tampered, trace_length_mismatch -> 100.00% -> 100.00% / 100.00%: 典型执行异常可完成仲裁; 4. 超时活性对照 -> no_recover vs recover -> 100.00% -> 0.00% 到 100.00%: 恢复决定仲裁是否完成;

Results : response_tampered 与 trace_length_mismatch 能进入 challenge + slashing 路径 ; attestation_invalid 、 da_unavailable 和 da_proof_invalid 在当前原型中可检测，但尚未统一进入罚没策略。

Analysis: 该结果强化了“检测到异常不等于完成仲裁”的主张。协议需要为不同故障定义不同结算语义，否则会把可检测性、可恢复性和可罚没性混为一谈。

Limitation: fault taxonomy 尚未形成生产级统一惩罚策略，未来需要区分直接拒绝、经济罚没、恢复后仲裁和人工治理介入的边界。

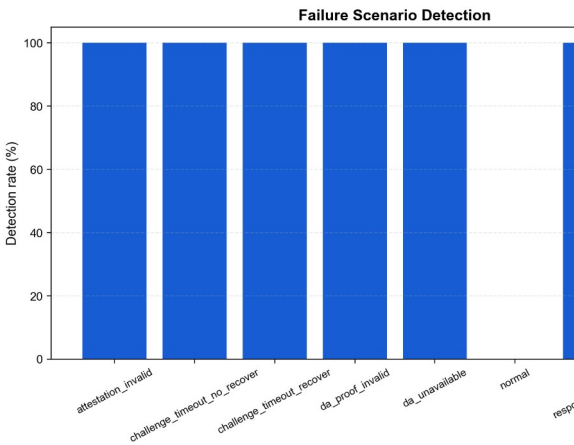


图 9 失败场景检测与仲裁结果图 9 失败场景检测与仲裁结果

7.7 RQ6: Solidity、本地 EVM 与 Sepolia 证据说明什么

Research Question: 原型协议是否具有链上状态推进对应物。

Hypothesis: 若 submit、challenge、recover、replay 和 resolve 均可映射为 Solidity 操作，则该机制至少具备本地链上可实现性。

Metrics: Hardhat 本地 EVM gasUsed、Sepolia chain id、部署区块和合约地址。

表 12 RQ6 本地 EVM gasUsed

操作	平均 gasUsed	解释
register DA	113526.00	记录 payload hash 与 DA root
submit rollup	299033.67	提交 compact commit
challenge open	283769.00	打开争议并加载证据
challenge respond	65886.00	被挑战方响应
challenge step	93807.00	执行二分定位步骤
challenge recover	156891.00	恢复超时挑战会话
challenge replay	135632.00	执行单步重放
challenge resolve	113402.00	结算争议
finalize	33477.00	争议窗口后正常最终确认

表 13 Sepolia 部署记录

项目	数值
Network / Chain ID	Sepolia / 11155111
部署区块	10825904
MockTEEVentifier	0xC42CB0Cf0D112Bd59E0f212F2DB200ca50a8b6A
MockDARegistry	0xc42C64a7De05bf3ad4f1c75CbD8E0506F7De99Ff
HybridTEERollup	0xB9B72f10bB8aBC1ed090f1B0443Fe57189c497Fd

Results: Solidity 原型在本地 EVM 上能够测量关键操作 gasUsed，并已部署到 Sepolia。

Analysis: 这些结果支持“协议具有链上对应物和可部署性”的阶段主张。`recover` 的本地 `gasUsed` 为 156891.00, 说明恢复不是零成本元数据操作, 而是明确的链上状态转移。

Limitation: 本地 EVM `gasUsed` 不能等同于 Sepolia 或主网费用; Sepolia 部署只能说明可部署性, 不能说明生产安全性、主网性能或真实 DA 网络行为。

7.7.1 Recovery Overhead Analysis

基于本地 EVM `gasUsed` 记录, 单次 `challenge` step 的 `gasUsed` 为 93807, `recover` 操作为 156891。因此, $\text{Recovery Overhead} \approx 156891 / 93807 \approx 1.67\times$ 。

`Recover` 操作虽然比单次 `challenge` step 开销更高, 但它属于低频异常路径操作, 对于正常路径吞吐量影响有限。其额外成本换取了挑战会话从停滞状态恢复到可结算状态的能力, 因此该开销在协议设计上是合理的。该分析仅针对原型环境, 不代表主网真实成本。

8 讨论与局限

本文的核心结论是: 在 Hybrid TEE-Rollup 中, 异常路径的安全性不应只由错误检测叙述。可追责系统还需要仲裁活性, 使已经打开的争议能够在超时后继续推进。`Recoverable challenge` 的意义在于把恢复定义为局部、可验证且不改写事实的协议操作。

该机制也有明确边界。第一, TEE 是模拟的, 不能外推到 SGX、TDX、CSV 或其他真实硬件环境。第二, DA 层是 `mock/verifiable registry`, 不具备 Celestia、EigenDA 等真实网络的经济安全。第三, 成本模型是配置化估算, 96.33% 只是模型趋势。第四, 本地 EVM `gasUsed` 是本地链基线, 不是主网费用。第五, Sepolia 部署只说明 `deployability evidence`, 不说明生产安全性。第六, 当前 `fault taxonomy` 尚未把所有可检测异常统一纳入罚没闭环。第七, `replay evidence` 仍部分由链下原型提供, 未来需要更完整的链上可验证执行环境或形式化证明。

尽管如此, 上述边界不削弱本文对局部协议机制的主张: 只要恢复操作保持挑战事实不变, 并由 DA 绑定与证据一致性检查约束争议上下文, 恢复就可以在不改变仲裁结果的前提下提升仲裁活性。

8.1 Threats to Validity

Internal Validity. 内部有效性威胁主要来自原型实现与真实系统之间的差异。模拟 TEE 只建模证明报告字段绑定, 不覆盖真实 SGX/TDX 的远程证明链、侧信道、I/O 完整性或硬件漏洞; 模拟 DA 注册表只验证 `payload hash`、Merkle root 和字段证明, 不覆盖真实 DA 网络中的采样、节点失效和经济安全。因此, 实验结果只能支持协议状态机与证据绑定逻辑, 而不能证明真实部署安全性。

Construct Validity. 构造有效性威胁主要来自指标解释。`challenge_success_rate`、`slashed_rate` 和 `recovery_success_rate` 衡量的是建模故障场景下的协议推进结果, 而不是开放网络中的长期可靠性。96.33% 降本来自配置化成本模型, 反映相同参数下的趋势比较; 本地 EVM `gasUsed` 衡量 Hardhat 环境中的函数调用开销, 不能代表主网费用、blob 市场价格或拥堵环境。

External Validity. 外部有效性威胁来自场景规模和部署环境。当前实验没有接入 Celestia、EigenDA、Espresso DA 等真实 DA 网络, 没有使用真实 TEE 硬件, 也没有在主网或大规模公开测试网流量下运行。因此, 结论应限定为初级研究原型对 RCP 机制的可行性验证, 而不是生产级 Rollup 性能或安全结论。

8.2 Future Work

未来工作首先需要接入真实 SGX/TDX 远程证明, 使 `proof_hash` 不仅绑定模拟 `attestation` 字段, 还能连接到硬件证明链和可审计的 `enclave identity`。其次, 需要接入 Celestia、EigenDA 或 Espresso DA 等真实 DA 网络, 评估 DA Binding 在真实数据发布、采样和恢复环境中的鲁棒性。

第三, 协议应与 FPVM 或其他链上可验证执行环境集成, 将当前由原型提供的 `replay evidence` 扩展为更强的单步执行证明。第四, 可使用 TLA+、Coq、Isabelle 或模型检查工具对状态转移函数、恢复不变性和终止性质进行形式化验证。第五, 跨链争议恢复值得进一步研究, 尤其是在提交链、DA 链和结算链分离的场景中如何保持挑战事实集合一致。第六, 多 `Watcher` 恢复机制需要引入激励、去重和冲突处理规则, 以避免单点看护者假设过强。

9 结论

本文面向 Hybrid TEE-Rollup 异常路径提出可恢复挑战协议, 将挑战会话建模为显式状态机, 并把超时恢复定义为保持挑战事实不变的活性操作。协议通

过 $F(cs)=F(\text{Recover}(cs))$ 、证据一致性和数据可用性绑定，避免恢复被滥用为证据替换或自动罚没入口。

理论上，本文给出恢复安全性、恢复活性和证据上下文一致性证明，说明恢复不会改变仲裁依据，并能在诚实看护者和证据可恢复条件下避免挑战永久停滞。实验上，研究原型显示恢复能够在建模超时场景中将挑战从 0% 完成推进到 100% 完成，二分定位轮次与 $\log_2(\text{trace steps})$ 一致，紧凑提交维持约 406 字节，配置化 DA 成本模型呈现结构性降本趋势。本地 EVM 和 Sepolia 记录进一步说明机制具有链上对应物和可部署性。

本文不声称实现生产级 Rollup、真实 TEE 安全或主网成本优势。其贡献在于提出并论证一种面向 Hybrid TEE-Rollup 异常路径的局部协议机制：RCP 在证据上下文不变的前提下提升仲裁活性，为后续接入真实 TEE、真实 DA 网络和更完整链上重放验证提供了可扩展基础。

本文最大的贡献并非实现某种具体 Rollup 系统，而是将挑战恢复从工程实现细节提升为具有形式化安全语义和活性目标的独立协议组件。通过 Fact Preservation、Evidence Consistency 和 Data Availability Binding，本文将 Arbitration Liveness 形式化为可分析、可证明、可恢复的协议属性。这一抽象为未来 Hybrid TEE-Rollup 异常路径设计提供了新的理论视角。

参考文献

- [1] WEN X, FENG Q, LYU H, et al. TEEROLLUP: Efficient Rollup Design Using Heterogeneous TEE[EB/OL]. arXiv:2409.14647, 2024.
- [2] PICCO G, FORTUGNO A. Dynamic Fraud Proofs[EB/OL]. arXiv:2502.10321, 2025.
- [3] AL-BASSAM M. LazyLedger: A Distributed Data Availability Ledger with Client-Side Smart Contracts[EB/OL]. arXiv:1905.09274, 2019.
- [4] TAS E N, TSE D, YANG L, et al. Light Clients for Lazy Blockchains[EB/OL]. arXiv:2203.15968, 2022.
- [5] Ethereum Foundation. EIP-4844: Shard Blob Transactions[EB/OL]. 2024.
- [6] Ethereum Foundation. Optimistic Rollups[EB/OL]. Ethereum Documentation, 2024.
- [7] Buterin V. An Incomplete Guide to Rollups[EB/OL]. 2021.
- [8] Kalodner H, Goldfeder S, Chen X, et al. Arbitrum: Scalable, Private Smart Contracts[EB/OL]. USENIX Security, 2018.
- [9] Optimism Collective. Fault Proofs and the OP Stack[EB/OL]. 2024.
- [10] Zhang F, Cecchetti E, Croman K, et al. Town Crier: An Authenticated Data Feed for Smart Contracts[C]. CCS, 2016.
- [11] Celestia Foundation. Celestia: A Modular Data Availability Network[EB/OL]. 2024.
- [12] Eigen Labs. EigenDA: Data Availability for Ethereum Rollups[EB/OL]. 2024.
- [13] Cartesi Foundation. Cartesi Rollups Documentation[EB/OL]. 2024.
- [14] opML Contributors. Optimistic Machine Learning on Blockchain[EB/OL]. 2024.
- [15] Offchain Labs. Arbitrum Nitro: A Second-generation Optimistic Rollup[EB/OL]. 2022.
- [16] Optimism Collective. OP Stack Technical Documentation[EB/OL]. 2024.
- [17] Espresso Systems. Espresso DA Documentation[EB/OL]. 2024.
- [18] Mustafa Al-Bassam. Sovereign Rollups[EB/OL]. 2022.
- [19] Matter Labs. zkSync Era Documentation[EB/OL]. 2024.
- [20] StarkWare. StarkNet Documentation[EB/OL]. 2024.
- [21] EigenLayer. EigenLayer Whitepaper[EB/OL]. 2023.
- [22] Cartesi Foundation. The Cartesi Machine and Cartesi Rollups Whitepaper[EB/OL]. 2023.
- [23] Intel Corporation. Intel Software Guard Extensions Developer Guide[EB/OL]. 2024.
- [24] Intel Corporation. Intel Trust Domain Extensions Architecture Specification[EB/OL]. 2024.
- [25] RISC Zero. RISC Zero zkVM Documentation[EB/OL]. 2024.
- [26] Succinct Labs. SP1 zkVM Documentation[EB/OL]. 2024.
- [27] Espresso Systems. Espresso Sequencer Documentation[EB/OL]. 2024.